Brain MRI Image Segmentation Using K-Means Clustering

Code

04 — Cross-Image Analysis and Cluster Transfer

Keman Xiang | PhD Application Portfolio

- Project Overview
  - Background and motivation
  - Research questions
  - Analysis roadmap
  - Data provenance
- 1. Import and Validate the MRI Images
  - 1.1 Read the original pixel matrices
  - 1.2 Check values, dimensions, and vectorization
- 2. Fit the Reference K-Means Model
  - 2.1 Intensity-only clustering
  - 2.2 Order clusters by centroid brightness
- 3. Transfer the Clusters to the Tumor Image
  - 3.1 Assign each pixel to its nearest fixed centroid
- 4. Visualize the Original and Segmented MRI Images
  - 4.1 Display the original grayscale images
  - 4.2 Display fixed-centroid segmentation maps
  - 4.3 Compare intensity-based reconstructions
- 5. Compare Cluster Occupancy Across Images
  - 5.1 Pixel counts and within-image proportions
  - 5.2 Visualize shared-cluster proportions
- 6. Evaluate Cross-Image Intensity Representation
  - 6.1 Reconstruction error using fixed healthy-image centers
  - 6.2 Examine how far observed intensities lie from their assigned centroids
- 7. Explore Intensity Distributions and Background Sensitivity
  - 7.1 Compare empirical grayscale distributions
  - 7.2 Check sensitivity to exact-zero background pixels
- 8. Discussion and Conclusions
  - 8.1 Main findings
  - 8.2 Limitations
  - 8.3 Conclusion and future directions
- 9. Reproducibility and References
  - Reproducibility notes
  - References and source materials

Project Overview

Project: Brain MRI Image Segmentation Using K-Means Clustering
Module: 04 — Cross-Image Analysis and Cluster Transfer
Environment: R / RStudio / R Markdown
Training image: `healthy.csv`
Application image: `tumor.csv`
Reference model: Five-cluster K-Means trained exclusively on the healthy image

Background and motivation

In this notebook, we will examine whether a grayscale segmentation model learned from one brain MRI can be applied to another image without refitting its cluster centers. We use pixel-intensity clustering to partition each image into groups of similar brightness. We first learn five intensity centroids from a healthy brain image; we then assign every pixel in an image containing a tumor to its closest **fixed healthy-image centroid**. We compare intensity distributions, cluster occupancy, reconstruction error, and spatial patterns of the resulting segmentation.

We distinguish **cross-image cluster assignment** from clinical tumor detection. We do not have pixel-level tissue annotations, a verified tumor mask, image-registration information, acquisition metadata, or patient-level labels. We therefore cannot estimate diagnostic accuracy or attribute differences between the images solely to the tumor. We treat the terms *healthy* and *tumor* as dataset identifiers rather than independently verified diagnoses.

Research questions

1. How do we assign pixels in the tumor image to clusters learned from the healthy image without refitting the centroids?
2. How do the two images differ in grayscale distribution and proportions assigned to each shared intensity cluster?
3. How well do fixed healthy-image centroids summarize intensities in the second image, relative to their fit on the training image?
4. Which conclusions can we draw from unsupervised intensity segmentation, and which clinical claims remain unsupported?

Analysis roadmap

Read and validate original CSVs → Fit healthy-image K-Means (k = 5) → Order centroids and transfer labels → Reconstruct both segmented images → Compare occupancy and intensity distributions → Evaluate fixed-centroid reconstruction → Discuss limitations.

Data provenance

Input	Role in our analysis
`healthy.csv`	Use the observed grayscale intensities to fit the five reference centroids.

Input	Role in our analysis
`tumor.csv`	Apply the fixed centroids to all observed pixels without fitting on this image.
`Unit6_Recitation.pdf`	Use the image-segmentation, K-Means, and medical-imaging context.
`Unit6_Recitation.R`	Use the matrix → vector → K-Means → new-image assignment workflow.
`week6_3.2.R`	Use the five-cluster reference, cross-image assignment concept, and visual comparison approach.

We use the CSVs as the **only sources of pixel data**. We do not synthesize pixels, substitute example images, or infer ground-truth tumor locations.

1. Import and Validate the MRI Images

1.1 Read the original pixel matrices

Read the source data.

Hide

```
find_data_dir <- function() {
  candidates <- c("data", "../data", ".", "..")
  present <- vapply(candidates, function(d) {
    file.exists(file.path(d, "healthy.csv")) &&
      file.exists(file.path(d, "tumor.csv"))
  }, logical(1))
  if (!any(present)) {
    stop("Cannot find healthy.csv and tumor.csv. Place both in data/.")
  }
  normalizePath(candidates[which(present)[1]], mustWork = TRUE)
}

data_dir <- find_data_dir()
healthy_df <- read.csv(file.path(data_dir, "healthy.csv"),
                       header = FALSE, check.names = FALSE)
tumor_df <- read.csv(file.path(data_dir, "tumor.csv"),
                     header = FALSE, check.names = FALSE)
healthy_matrix <- as.matrix(healthy_df)
tumor_matrix <- as.matrix(tumor_df)
```

Code explanation. We convert the CSV data frames into matrices without changing their row or column order. We preserve the original image dimensions because we will later reconstruct cluster assignments in precisely the same arrangement.

1.2 Check values, dimensions, and vectorization

Verify that both matrices are numeric, contain finite values, and use the expected grayscale interval $[0, 1]$. Flatten each matrix using R's column-major ordering and confirm that reshaping the vectors preserves every pixel value. We ignore matrix names during the equality check because CSV-derived column names do not represent image intensities.

Hide

```
validate_image <- function(x, name) {
  stopifnot(is.matrix(x), is.numeric(x), nrow(x) > 0L, ncol(x) > 0L)
  if (anyNA(x) || any(!is.finite(x))) {
    stop(name, " contains missing or non-finite pixel values.")
  }
  if (any(x < 0 | x > 1)) {
    stop(name, " contains intensities outside [0, 1].")
  }
  invisible(TRUE)
}
validate_image(healthy_matrix, "Healthy MRI")
validate_image(tumor_matrix, "Tumor MRI")

healthy_vector <- as.vector(healthy_matrix)
tumor_vector <- as.vector(tumor_matrix)
stopifnot(
  length(healthy_vector) == prod(dim(healthy_matrix)),
  length(tumor_vector) == prod(dim(tumor_matrix)),
  isTRUE(all.equal(unname(matrix(healthy_vector,
                                 nrow = nrow(healthy_matrix),
                                 ncol = ncol(healthy_matrix))),
                   unname(healthy_matrix), check.attributes = FALSE)),
  isTRUE(all.equal(unname(matrix(tumor_vector,
                                 nrow = nrow(tumor_matrix),
                                 ncol = ncol(tumor_matrix))),
                   unname(tumor_matrix), check.attributes = FALSE))
)

image_info <- data.frame(
  Image = c("Healthy", "Tumor"),
  Rows = c(nrow(healthy_matrix), nrow(tumor_matrix)),
  Columns = c(ncol(healthy_matrix), ncol(tumor_matrix)),
  Pixels = c(length(healthy_vector), length(tumor_vector)),
  Mean = c(mean(healthy_vector), mean(tumor_vector)),
  SD = c(sd(healthy_vector), sd(tumor_vector)),
  Zero_Percent = 100 * c(mean(healthy_vector == 0), mean(tumor_vector ==
0))
)
knitr::kable(image_info, digits = 4,
             caption = "Observed image dimensions and grayscale summarie
s")
```

Observed image dimensions and grayscale summaries

Image	Rows	Columns	Pixels	Mean	SD	Zero_Percent
Healthy	566	646	365636	0.1302	0.1257	9.2950
Tumor	571	512	292352	0.1595	0.1715	34.6692

Output interpretation. We observe 365,636 healthy-image pixels and 292,352 tumor-image pixels. We calculate all values directly from the uploaded CSVs. We observe mean intensities of 0.1302 and 0.1595, respectively. We do not interpret the mean difference as a tumor effect: we have different image dimensions, backgrounds, and potentially different imaging conditions.

2. Fit the Reference K-Means Model

2.1 Intensity-only clustering

We represent each pixel by a single grayscale value $x_i \in [0, 1]$. We minimize the within-cluster sum of squares for $k = 5$:

$$\text{WCSS} = \sum_{j=1}^{5} \sum_{i \in C_j} (x_i - \mu_j)^2,$$

where μ_j is the mean intensity of cluster j. We use `set.seed(1)` and `iter.max = 1000` for a reproducible five-cluster reference fit. We do **not** include the tumor-image pixels when estimating the centers.

Hide

```
k <- 5L
set.seed(1)
healthy_kmeans <- kmeans(healthy_vector, centers = k, iter.max = 1000)

stopifnot(
  length(healthy_kmeans$cluster) == length(healthy_vector),
  sum(healthy_kmeans$size) == length(healthy_vector),
  nrow(healthy_kmeans$centers) == k
)

raw_centers <- as.numeric(healthy_kmeans$centers[, 1])
stopifnot(length(unique(raw_centers)) == k, all(is.finite(raw_centers)))

reference_fit <- data.frame(
  Original_Cluster = seq_len(k),
  Center = raw_centers,
  Size = as.integer(healthy_kmeans$size),
  Proportion = as.numeric(healthy_kmeans$size) / length(healthy_vector)
)
knitr::kable(reference_fit, digits = 5,
             caption = "Reference K-Means clusters before ordering by brig
htness")
```

Reference K-Means clusters before ordering by brightness

Original_Cluster	Center	Size	Proportion

Original_Cluster	Center	Size	Proportion
1	0.11656	114480	0.31310
2	0.20136	65557	0.17930
3	0.49139	18298	0.05004
4	0.32991	27338	0.07477
5	0.02189	139963	0.38279

Code explanation. We fit the model once on the healthy-image pixel vector. We inspect the five estimated centers and the number of pixels assigned to each cluster. We treat cluster numbers returned by `kmeans()` as arbitrary identifiers: cluster 1 is not inherently darker or brighter than cluster 2.

Output interpretation. We obtain five centroids spanning the grayscale distribution of the healthy image. We use the numerical center values—not their original cluster IDs—to define a consistent brightness ordering for both images. We cannot infer anatomical tissue identity from the centroids alone.

2.2 Order clusters by centroid brightness

We assign a stable rank from 1 (darkest) to 5 (brightest). Map the healthy-image K-Means assignments into this ordered label system and retain the same order when labeling the tumor image. We use the **same five colors** across both images to make comparisons meaningful.

Hide

```
        order_index <- order(raw_centers)
        ordered_centers <- raw_centers[order_index]
        original_to_rank <- integer(k)
        original_to_rank[order_index] <- seq_len(k)

        healthy_labels <- original_to_rank[healthy_kmeans$cluster]
        stopifnot(length(healthy_labels) == length(healthy_vector),
                  all(healthy_labels %in% seq_len(k)))

        centroid_table <- data.frame(
          Brightness_Rank = seq_len(k),
          Center = ordered_centers,
          Healthy_Pixels = tabulate(healthy_labels, nbins = k)
        )
        centroid_table$Healthy_Percent <- 100 * centroid_table$Healthy_Pixels /
          length(healthy_vector)
        knitr::kable(centroid_table, digits = 5,
                     caption = "Five fixed healthy-image centroids, ordered dark t
o bright")
```

Five fixed healthy-image centroids, ordered dark to bright

Brightness_Rank	Center	Healthy_Pixels	Healthy_Percent
1	0.02189	139963	38.27933
2	0.11656	114480	31.30983
3	0.20136	65557	17.92958
4	0.32991	27338	7.47683
5	0.49139	18298	5.00443

Output interpretation. We obtain ordered centers from 0.0219 to 0.4914. We use these values as fixed reference intensities in every subsequent comparison. We keep the centroids unchanged when we move to the second image.

3. Transfer the Clusters to the Tumor Image

3.1 Assign each pixel to its nearest fixed centroid

We assign each tumor-image pixel to the nearest healthy-image centroid under squared Euclidean distance:

$$\hat{c}(x) = \arg \min_{j \in \{1,\ldots,5\}} (x - \mu_j)^2.$$

We do **not** run `kmeans()` on `tumor_vector`, because that would estimate new centers and would no longer test transfer of the fixed healthy-image representation. We implement the one-dimensional nearest-centroid rule directly in base R. With sorted centers, the decision boundaries are the midpoints between adjacent centers. This approach avoids installing additional packages during knitting while implementing the same closest-centroid assignment principle used in K-Means prediction.

Hide

```
        # For one-dimensional Euclidean distances, nearest-centroid regions
        # are separated by midpoints between consecutive sorted centers.
        boundaries <- (head(ordered_centers, -1L) +
                       tail(ordered_centers, -1L)) / 2

        assign_fixed_centroids <- function(values, cutpoints) {
          stopifnot(is.numeric(values), all(is.finite(values)),
                    is.numeric(cutpoints), all(diff(cutpoints) > 0))
          # findInterval returns indices 0, 1, ..., k-1; add 1 for labels 1..k.
          as.integer(findInterval(values, cutpoints) + 1L)
        }

        tumor_labels <- assign_fixed_centroids(tumor_vector, boundaries)
        stopifnot(length(tumor_labels) == length(tumor_vector),
                  all(tumor_labels %in% seq_len(k)))

        # Verify nearest-centroid assignment against direct squared distances
        # on all distinct observed tumor intensities (not just a sample).
        unique_tumor_values <- sort(unique(tumor_vector))
        assigned_unique <- assign_fixed_centroids(unique_tumor_values, boundaries)
        direct_unique <- vapply(unique_tumor_values, function(v) {
          which.min((v - ordered_centers)^2)
        }, integer(1))
        stopifnot(identical(assigned_unique, direct_unique))

        boundary_table <- data.frame(
          Boundary = paste0("Cluster ", seq_len(k - 1L), " / ", seq.int(2L, k)),
          Midpoint = boundaries
        )
        knitr::kable(boundary_table, digits = 6,
                     caption = "Fixed decision boundaries between adjacent intensi
ty clusters")
```

Fixed decision boundaries between adjacent intensity clusters

Boundary	Midpoint
Cluster 1 / 2	0.069221
Cluster 2 / 3	0.158960
Cluster 3 / 4	0.265639
Cluster 4 / 5	0.410650

Code explanation. We use the fixed midpoints to classify each tumor-image pixel. We independently verify the result against explicit nearest-centroid distance calculations for **every distinct intensity value observed in `tumor.csv`** . We do not require the images to have matching dimensions because we assign each pixel independently by its intensity.

Output interpretation. We assign all 292,352 tumor-image pixels to one of the five reference clusters. We retain the same intensity thresholds and labels across images. We should interpret these assignments as brightness categories, not predicted tumor probabilities or anatomical diagnoses.

4. Visualize the Original and Segmented MRI Images

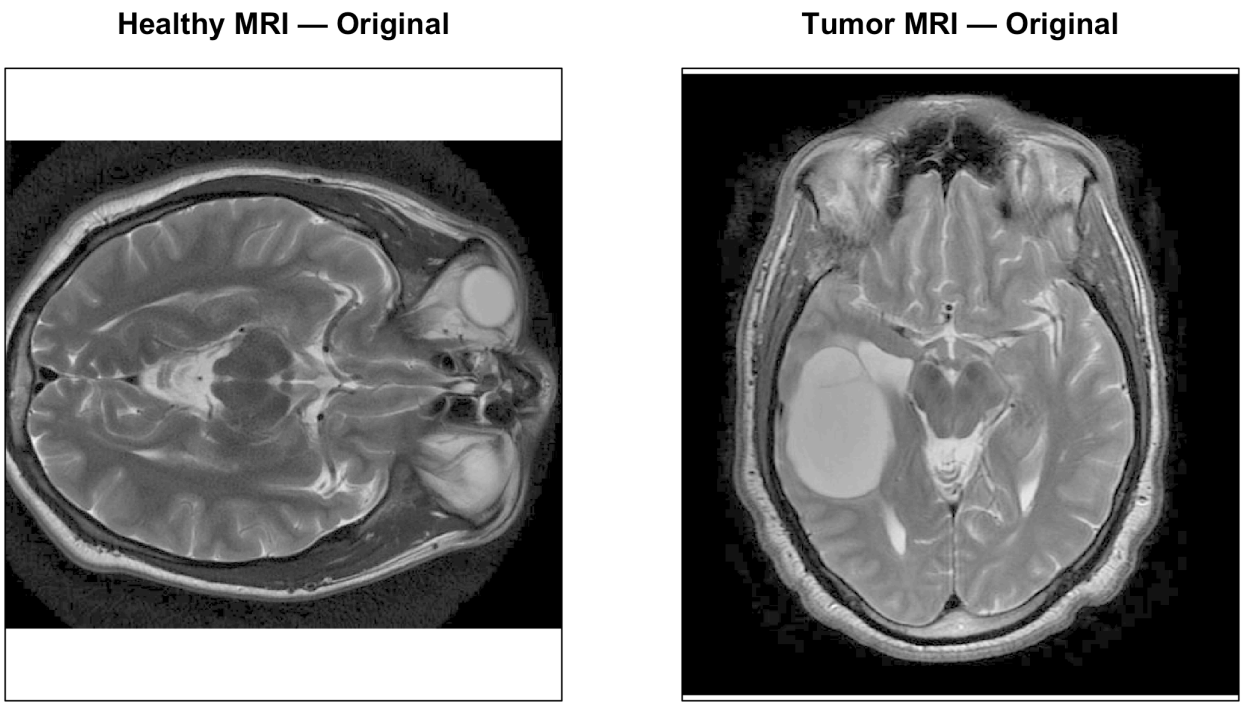
4.1 Display the original grayscale images

Here, we use the same display transformation for both matrices. We transpose and reverse the row axis for `image()` so the first CSV row appears at the top and the first CSV column appears at the left. We preserve each image's original pixel arrangement and use a fixed grayscale range of 0–1.

Hide

```
plot_pixel_matrix <- function(mat, palette, zlim, main) {
  # R's image() uses the first matrix dimension as horizontal;
  # transpose and reverse rows to show CSV row 1 at the top.
  display_mat <- t(mat[nrow(mat):1L, , drop = FALSE])
  image(x = seq_len(nrow(display_mat)),
        y = seq_len(ncol(display_mat)),
        z = display_mat, col = palette, zlim = zlim,
        axes = FALSE, xlab = "", ylab = "", main = main,
        useRaster = TRUE, asp = 1)
  box()
}

op <- par(mfrow = c(1, 2), mar = c(2, 2, 3, 2))
plot_pixel_matrix(healthy_matrix, gray.colors(256, start = 0, end = 1),
                  c(0, 1), "Healthy MRI — Original")
plot_pixel_matrix(tumor_matrix, gray.colors(256, start = 0, end = 1),
                  c(0, 1), "Tumor MRI — Original")
```



Original healthy and tumor MRI images reconstructed from the uploaded grayscale CSV matrices.

Hide

```
par(op)
```

Output interpretation. We can visually inspect broad anatomical and background patterns in the two original images. We do not assume pixel-wise spatial correspondence: the images have different dimensions and are not registered to each other.

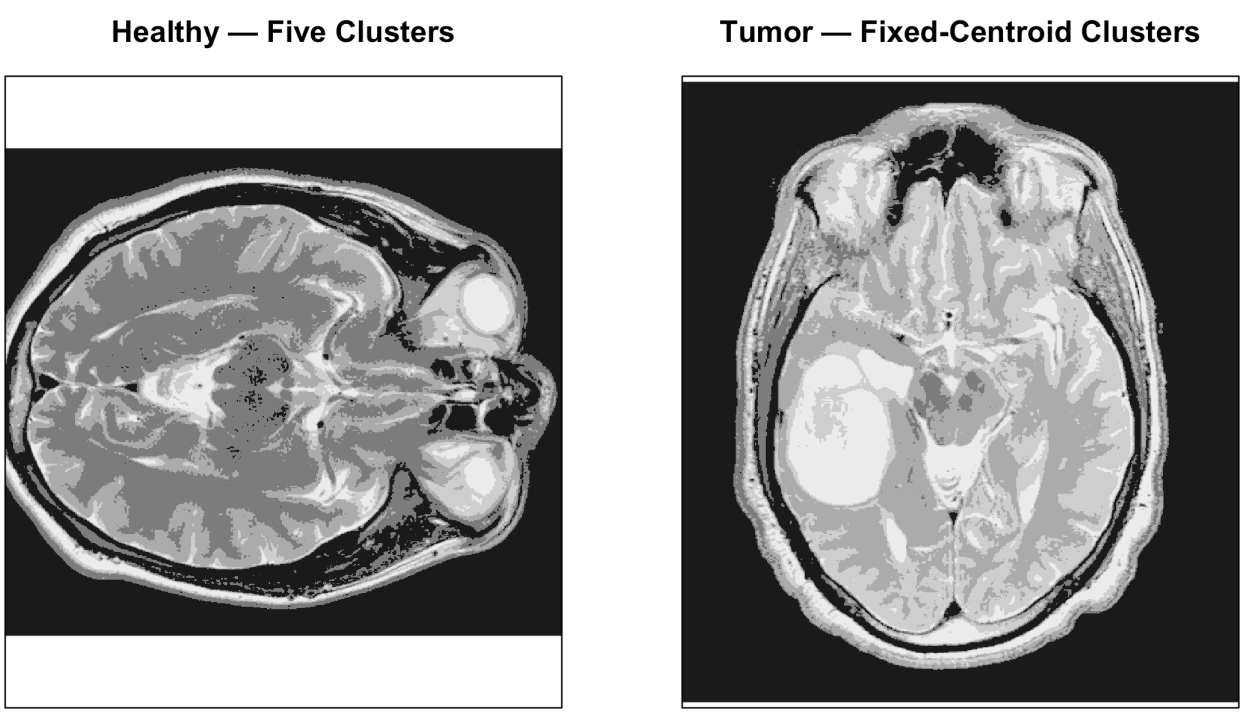
4.2 Display fixed-centroid segmentation maps

We reshape the ordered labels into the original dimensions and plot both images using the same discrete palette. We display five intensity classes, with darker colors representing lower-intensity centroids and lighter colors representing higher-intensity centroids.

Hide

```
healthy_label_matrix <- matrix(healthy_labels,
                               nrow = nrow(healthy_matrix),
                               ncol = ncol(healthy_matrix))
tumor_label_matrix <- matrix(tumor_labels,
                             nrow = nrow(tumor_matrix),
                             ncol = ncol(tumor_matrix))

# A monotone grayscale palette makes intensity rank interpretable.
cluster_palette <- gray.colors(k, start = 0.12, end = 0.92)
op <- par(mfrow = c(1, 2), mar = c(2, 2, 3, 2))
plot_pixel_matrix(healthy_label_matrix, cluster_palette,
                  c(0.5, k + 0.5), "Healthy — Five Clusters")
plot_pixel_matrix(tumor_label_matrix, cluster_palette,
                  c(0.5, k + 0.5), "Tumor — Fixed-Centroid Clusters")
```



Five-class intensity segmentation of both MRIs using identical centroids fitted on the healthy image.

Hide

```
par(op)
```

Code explanation. We reconstruct each segmentation map using the original image dimensions. We use identical centroids, class boundaries, and colors, so cluster rank 3 denotes the same intensity range in both panels. We do not align, resize, or interpolate the input images.

Output interpretation. We can compare how similar intensity categories appear in different spatial arrangements. We may observe visually distinct regions in the tumor image, therefore we can suspect the existence and region of a tumor formation. However, for reasons of discretion, we cannot designate that

cluster as a tumor region definitely without ground-truth annotation or clinical validation. We also cannot conclude that a difference in appearance is caused by pathology rather than image background or acquisition conditions.

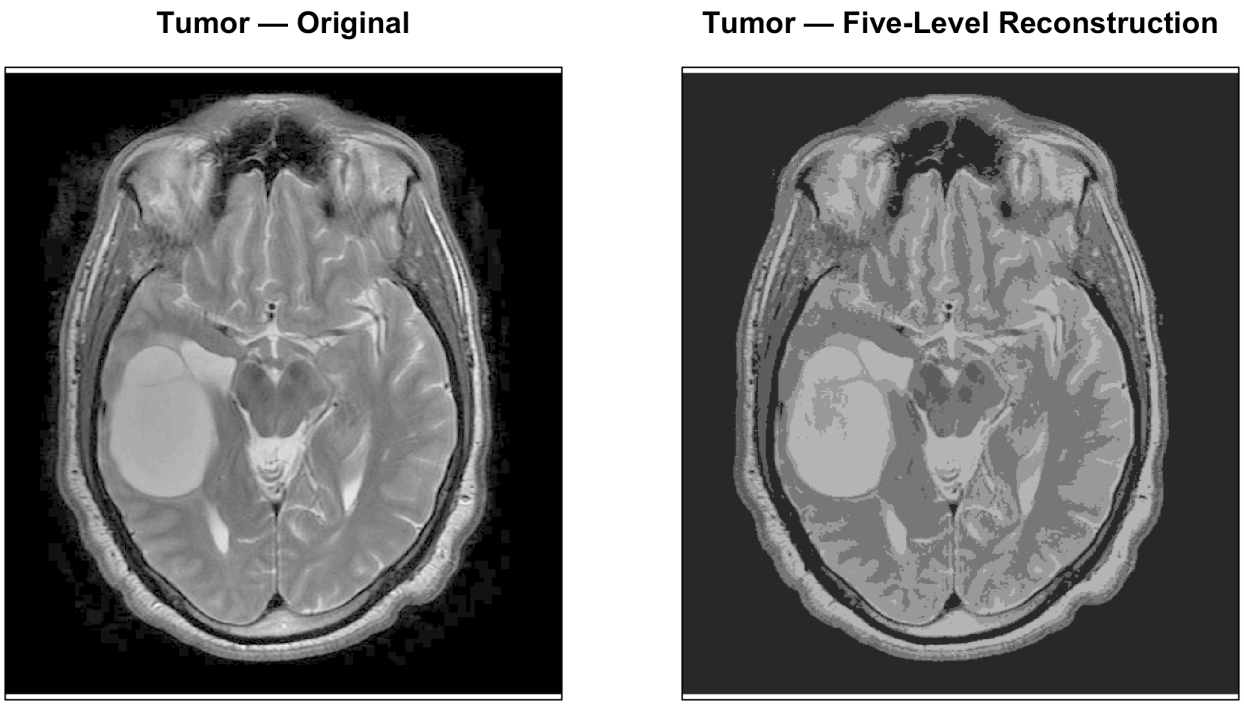
4.3 Compare intensity-based reconstructions

We replace each pixel by the centroid of its assigned cluster. We use this quantized image to examine the information retained by the five-level representation. We keep the original intensity range for both reconstructed panels.

Hide

```
healthy_reconstructed <- matrix(ordered_centers[healthy_labels],
                                nrow = nrow(healthy_matrix),
                                ncol = ncol(healthy_matrix))
tumor_reconstructed <- matrix(ordered_centers[tumor_labels],
                              nrow = nrow(tumor_matrix),
                              ncol = ncol(tumor_matrix))

op <- par(mfrow = c(1, 2), mar = c(2, 2, 3, 2))
plot_pixel_matrix(tumor_matrix, gray.colors(256, 0, 1), c(0, 1),
                  "Tumor — Original")
plot_pixel_matrix(tumor_reconstructed, gray.colors(256, 0, 1), c(0, 1),
                  "Tumor — Five-Level Reconstruction")
```



Original tumor MRI and its five-level reconstruction using fixed healthy-image intensity centroids.

Hide

```
        par(op)
```

Output interpretation. We preserve coarse grayscale structure while reducing the number of represented intensities to five. We expect some detail to disappear because every original pixel is replaced by one of the five healthy-image centroids. We treat this as intensity quantization, not anatomical segmentation accuracy.

5. Compare Cluster Occupancy Across Images

5.1 Pixel counts and within-image proportions

We then compare the proportion of pixels assigned to each fixed intensity class. We use percentages rather than raw counts for the primary comparison because the images contain different numbers of pixels.

Hide

```
        healthy_counts <- tabulate(healthy_labels, nbins = k)
        tumor_counts <- tabulate(tumor_labels, nbins = k)
        healthy_pct <- 100 * healthy_counts / length(healthy_labels)
        tumor_pct <- 100 * tumor_counts / length(tumor_labels)

        occupancy <- data.frame(
          Cluster = seq_len(k),
          Healthy_Centroid = ordered_centers,
          Healthy_Pixels = healthy_counts,
          Tumor_Pixels = tumor_counts,
          Healthy_Percent = healthy_pct,
          Tumor_Percent = tumor_pct,
          Difference_pp = tumor_pct - healthy_pct
        )
        stopifnot(sum(healthy_counts) == length(healthy_vector),
                  sum(tumor_counts) == length(tumor_vector),
                  abs(sum(healthy_pct) - 100) < 1e-8,
                  abs(sum(tumor_pct) - 100) < 1e-8)
        knitr::kable(occupancy, digits = 3,
                     caption = "Pixel occupancy by shared healthy-image intensity
cluster")
```

Pixel occupancy by shared healthy-image intensity cluster

Cluster	Healthy_Centroid	Healthy_Pixels	Tumor_Pixels	Healthy_Percent	Tumor_Percent	Difference_pp
1	0.022	139963	135357	38.279	46.299	8.020
2	0.117	114480	17731	31.310	6.065	-25.245
3	0.201	65557	58506	17.930	20.012	2.083
4	0.330	27338	52220	7.477	17.862	10.385

Cluster	Healthy_Centroid	Healthy_Pixels	Tumor_Pixels	Healthy_Percent	Tumor_Percent	Difference_pp
5	0.491	18298	28538	5.004	9.762	4.757

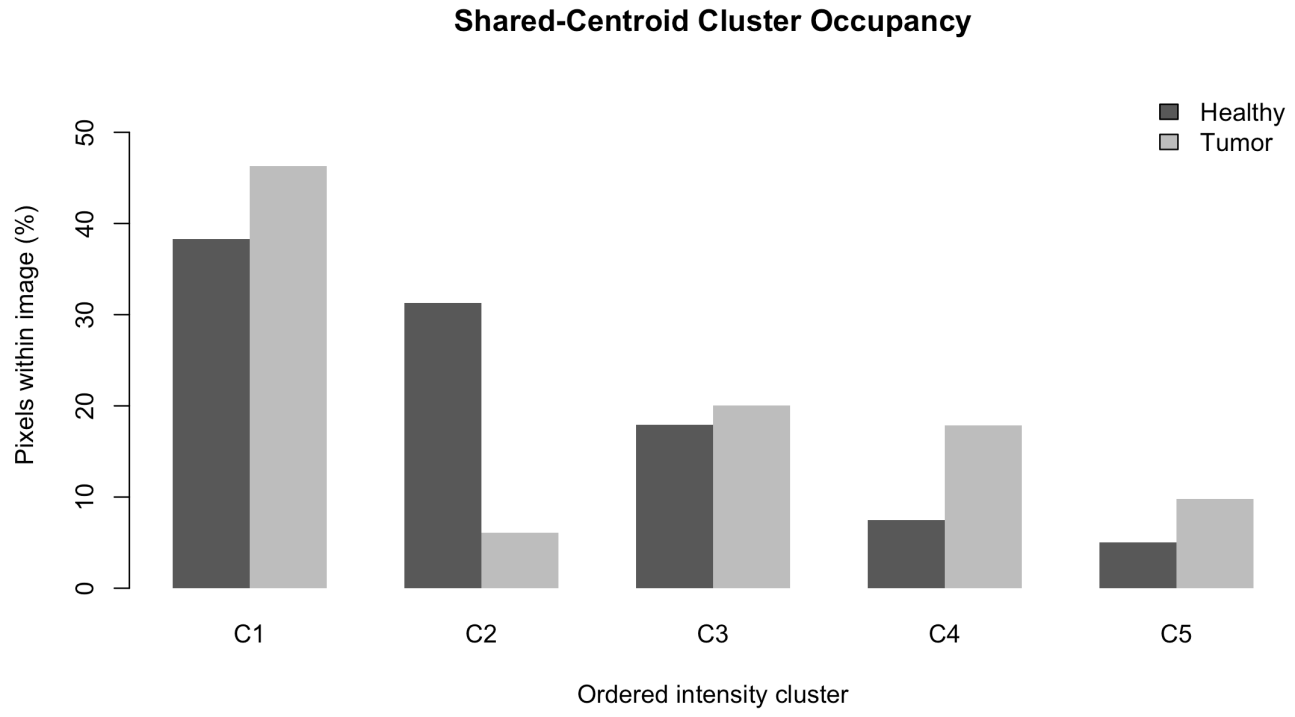
Code explanation. We calculate counts and within-image percentages for each of the five ordered intensity clusters. We report differences in **percentage points**, computed as tumor-image percentage minus healthy-image percentage. We do not use differences in raw pixel counts as evidence because image sizes differ.

Output interpretation. We find that cluster 2 has the largest absolute change in pixel share (-25.24 percentage points). We interpret this as a change in the distribution of grayscale categories across the two images. Due to carefulness , we cannot attribute that shift to tumor tissue specifically without clinical examination.

5.2 Visualize shared-cluster proportions

Hide

```
        barplot(rbind(healthy_pct, tumor_pct), beside = TRUE,
                names.arg = paste0("C", seq_len(k)),
                col = c("gray35", "gray75"), border = NA,
                ylim = c(0, max(c(healthy_pct, tumor_pct)) * 1.2),
                ylab = "Pixels within image (%)", xlab = "Ordered intensity cluste
r",
                main = "Shared-Centroid Cluster Occupancy")
        legend("topright", legend = c("Healthy", "Tumor"),
               fill = c("gray35", "gray75"), bty = "n")
```



Proportion of pixels assigned to each fixed intensity cluster in the two MRI images.

Output interpretation. We compare the relative frequency of shared intensity categories, not the location of homologous tissues. We note that differences may reflect background coverage, field of view, imaging parameters, or anatomical variation as well as pathology.

6. Evaluate Cross-Image Intensity Representation

6.1 Reconstruction error using fixed healthy-image centers

We evaluate how closely the assigned centroids approximate each original pixel intensity. We compute mean absolute error (MAE), mean squared error (MSE), and root mean squared error (RMSE):

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|x_i - \hat{x}_i|, \quad \mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2, \quad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}.$$

We compute the same metrics for both images using the same five healthy-image centers. We normalize by the number of pixels so the results are comparable despite unequal image dimensions.

Hide

```
intensity_metrics <- function(actual, labels, centers) {
  predicted <- centers[labels]
  errors <- actual - predicted
  c(MAE = mean(abs(errors)),
    MSE = mean(errors^2),
    RMSE = sqrt(mean(errors^2)),
    WCSS = sum(errors^2))
}

healthy_error <- intensity_metrics(healthy_vector, healthy_labels,
                                   ordered_centers)
tumor_error <- intensity_metrics(tumor_vector, tumor_labels,
                                 ordered_centers)

# The healthy fit should reproduce its own reported within-cluster SSE.
stopifnot(isTRUE(all.equal(unname(healthy_error["WCSS"]),
                           unname(healthy_kmeans$tot.withinss),
                           tolerance = 1e-7)))

error_table <- data.frame(
  Image = c("Healthy (training)", "Tumor (transfer)"),
  MAE = c(healthy_error["MAE"], tumor_error["MAE"]),
  MSE = c(healthy_error["MSE"], tumor_error["MSE"]),
  RMSE = c(healthy_error["RMSE"], tumor_error["RMSE"]),
  WCSS = c(healthy_error["WCSS"], tumor_error["WCSS"])
)
knitr::kable(error_table, digits = 6,
             caption = "Fixed-centroid intensity reconstruction errors")
```

Fixed-centroid intensity reconstruction errors

Image	MAE	MSE	RMSE	WCSS
Healthy (training)	0.022047	0.000826	0.028746	302.1291
Tumor (transfer)	0.029319	0.001573	0.039661	459.8580

Code explanation. We replace each pixel with its assigned centroid and calculate the residual intensity. We verify that the healthy-image total squared error matches the K-Means model's reported within-cluster sum of squares. We report MAE, MSE, and RMSE per pixel, while noting that total WCSS depends on image size.

Output interpretation. We obtain a healthy-image RMSE of 0.02875 and a tumor-image RMSE of 0.03966. We interpret any difference as a change in the numerical fit of the **same intensity codebook** to two different images. We do not interpret a lower reconstruction error as evidence of better tumor identification or a higher error as proof of pathology.

6.2 Examine how far observed intensities lie from their assigned centroids

We compare the distribution of absolute quantization residuals. We also report the fraction of pixels outside the span of the reference centroids. We treat the latter as a descriptive indicator of how much of each image lies beyond the fitted center range—not as an anomaly detector.

Hide

```
        healthy_abs_error <- abs(healthy_vector - ordered_centers[healthy_labels])
        tumor_abs_error <- abs(tumor_vector - ordered_centers[tumor_labels])

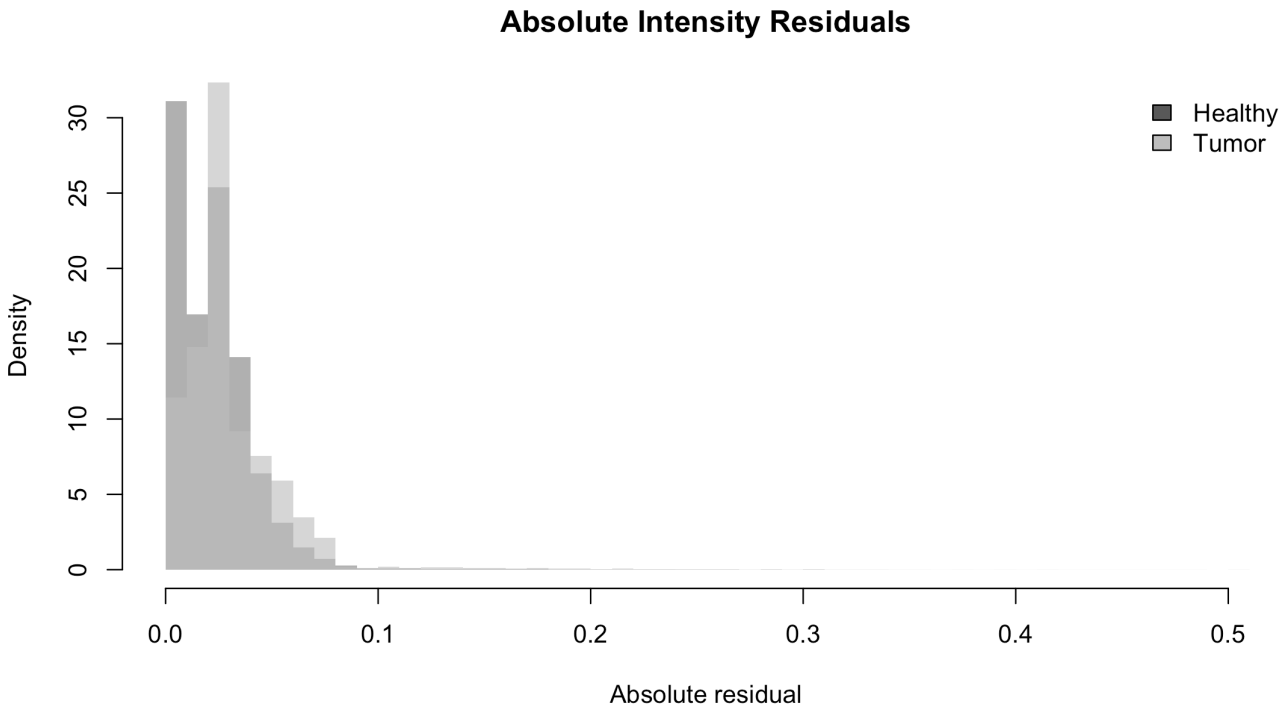
        residual_summary <- data.frame(
          Image = c("Healthy", "Tumor"),
          Median_Abs_Error = c(median(healthy_abs_error), median(tumor_abs_erro
r)),
          P90_Abs_Error = c(unname(quantile(healthy_abs_error, 0.90)),
                            unname(quantile(tumor_abs_error, 0.90))),
          Beyond_Centroid_Range_Percent = 100 * c(
            mean(healthy_vector < min(ordered_centers) |
                   healthy_vector > max(ordered_centers)),
            mean(tumor_vector < min(ordered_centers) |
                   tumor_vector > max(ordered_centers)))
        )
        knitr::kable(residual_summary, digits = 5,
                     caption = "Absolute reconstruction residual diagnostics")
```

Absolute reconstruction residual diagnostics

Image	Median_Abs_Error	P90_Abs_Error	Beyond_Centroid_Range_Percent
Healthy	0.02085	0.04223	25.6356
Tumor	0.02189	0.05540	46.4382

Hide

```
        breaks_common <- seq(0, 1, by = 0.01)
        hist(healthy_abs_error, breaks = breaks_common, freq = FALSE,
             col = grDevices::adjustcolor("gray35", alpha.f = 0.5),
             border = NA, xlim = c(0, max(c(healthy_abs_error, tumor_abs_error))),
             main = "Absolute Intensity Residuals", xlab = "Absolute residual")
        hist(tumor_abs_error, breaks = breaks_common, freq = FALSE,
             col = grDevices::adjustcolor("gray75", alpha.f = 0.6),
             border = NA, add = TRUE)
        legend("topright", legend = c("Healthy", "Tumor"),
               fill = c("gray35", "gray75"), bty = "n")
```



Distribution of absolute intensity residuals under the same five fixed healthy-image centroids.

Output interpretation. We use the residual distributions to assess how efficiently the healthy-image intensity codebook summarizes each image. We recognize that the two samples have different backgrounds and grayscale frequencies; a distribution shift is not automatically a tumor-specific signal.

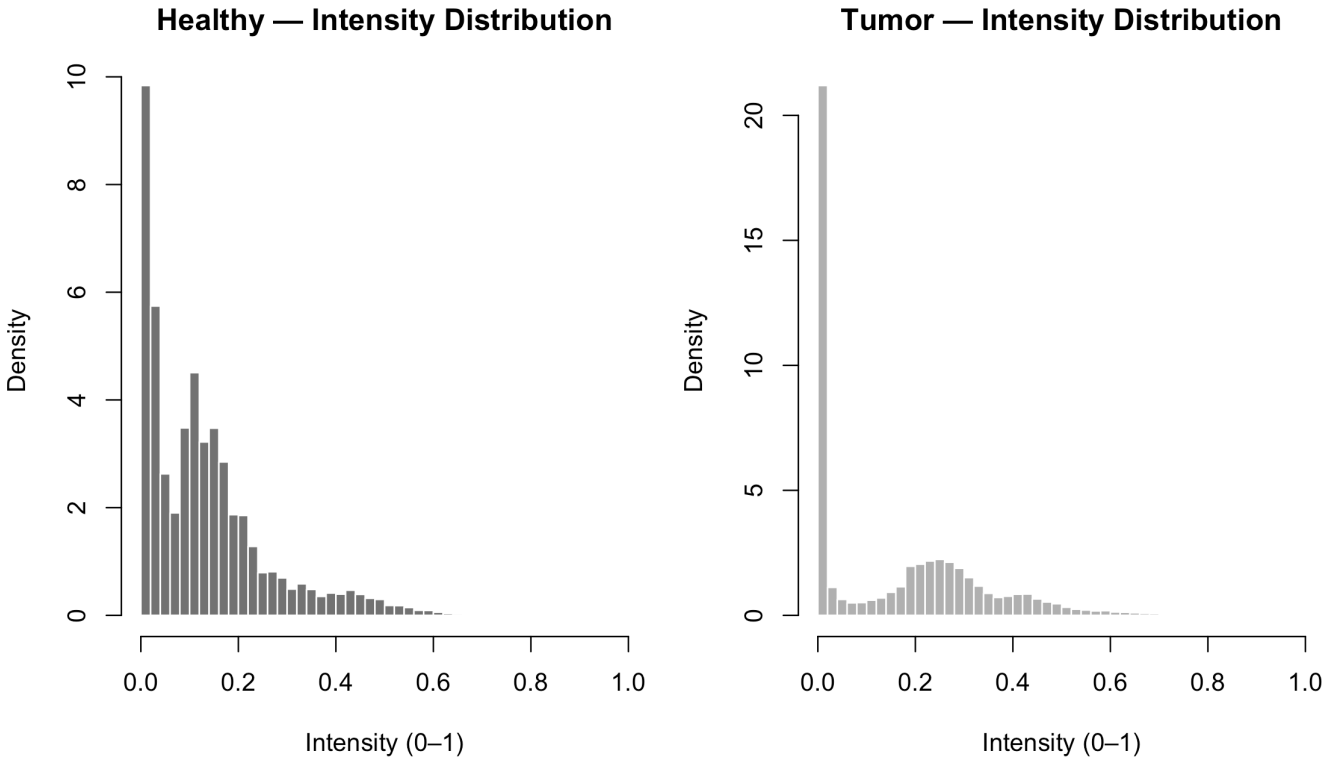
7. Explore Intensity Distributions and Background Sensitivity

7.1 Compare empirical grayscale distributions

We compare the original pixel intensities using equal-width bins. We normalize the histograms to density to avoid confounding distribution shape with pixel count.

Hide

```
op <- par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
hist(healthy_vector, breaks = seq(0, 1, length.out = 51),
     probability = TRUE, col = "gray45", border = "white",
     main = "Healthy – Intensity Distribution", xlab = "Intensity (0–1)",
     xlim = c(0, 1))
hist(tumor_vector, breaks = seq(0, 1, length.out = 51),
     probability = TRUE, col = "gray70", border = "white",
     main = "Tumor – Intensity Distribution", xlab = "Intensity (0–1)",
     xlim = c(0, 1))
```



Original grayscale-intensity distributions of healthy and tumor MRI pixels.

Hide

```
par(op)
```

Output interpretation. We examine differences in the frequency of dark and bright pixels. We interpret these as whole-image intensity-distribution differences, which may be influenced strongly by background pixels and differences in image framing.

7.2 Check sensitivity to exact-zero background pixels

We separately summarize exact-zero and positive-intensity pixels. We use this as a **sensitivity analysis**, not a definitive brain-tissue mask: we cannot assume that every zero-intensity pixel is background or that every positive pixel is tissue.

Hide

```
        positive_summary <- function(x, name) {
          positive <- x[x > 0]
          data.frame(Image = name,
                     Zero_Percent = 100 * mean(x == 0),
                     Positive_Pixels = length(positive),
                     Positive_Mean = if (length(positive)) mean(positive) else NA_
real_,
                     Positive_Median = if (length(positive)) median(positive) else
NA_real_)
        }
        zero_table <- rbind(positive_summary(healthy_vector, "Healthy"),
                            positive_summary(tumor_vector, "Tumor"))
        knitr::kable(zero_table, digits = 5,
                     caption = "Exact-zero sensitivity and positive-intensity summ
aries")
```

Exact-zero sensitivity and positive-intensity summaries

Image	Zero_Percent	Positive_Pixels	Positive_Mean	Positive_Median
Healthy	9.29504	331650	0.14358	0.11966
Tumor	34.66917	190996	0.24417	0.24314

Output interpretation. We observe exact-zero shares of 9.30% in the healthy image and 34.67% in the tumor image. We therefore treat all-pixel occupancy differences with caution. We avoid claiming that excluding zeros produces a validated tissue-only analysis.

8. Discussion and Conclusions

8.1 Main findings

We derive five grayscale centroids from the healthy image and apply them without refitting to every pixel in the tumor image. Then, we verify the cross-image assignment against explicit nearest-centroid calculations on all distinct observed tumor intensities. We reconstruct both segmentation maps using a common brightness ordering and palette, and we quantify cross-image differences using pixel proportions and per-pixel reconstruction errors.

We find that the largest absolute cluster-share change is 25.24 percentage points. We obtain transfer RMSE 0.03966 compared with training RMSE 0.02875. We interpret these quantities as **descriptive intensity-composition and quantization results**, not evidence of clinical tumor localization.

8.2 Limitations

- **Use grayscale intensity only.** We do not model neighboring pixels, texture, spatial coherence, or anatomical structure.

- **Analyze only two images.** We cannot estimate between-patient variability or generalization across scanners, protocols, or populations.
- **Have no ground-truth segmentation masks.** We cannot calculate Dice, intersection-over-union, sensitivity, specificity, or tumor-detection accuracy.
- **Do not register the images.** We compare distributions and visual patterns, not anatomically matched pixels.
- **Do not identify a tumor-specific cluster.** We do not equate a bright, dark, or unusual-intensity cluster with pathology.
- **Do not adjust for acquisition differences.** We cannot separate biological effects from scanner, contrast, preprocessing, cropping, or background differences.
- **Choose five clusters as a fixed reference.** We do not claim that this value is clinically optimal.

8.3 Conclusion and future directions

We demonstrate a reproducible cross-image intensity-segmentation workflow using K-Means centers estimated on one MRI and transferred unchanged to another. We show how fixed centroids support consistent labeling, comparable occupancy summaries, and transparent quantization-error diagnostics. We regard this as an exploratory representation of grayscale image structure.

We would need verified anatomical labels, larger multi-image datasets, standardized preprocessing, spatial features, and clinically appropriate evaluation before studying tumor localization or medical usefulness. We could also compare intensity-only K-Means with spatially aware or probabilistic segmentation methods in future work.

9. Reproducibility and References

Reproducibility notes

Read both source CSVs directly and use `set.seed(1)` before fitting the reference K-Means model. We rely on base R for clustering, nearest-centroid assignment, validation, and plotting, and on `knitr / rmarkdown` for document rendering. We do not install packages from within the report. We use fixed centroids for the tumor image and avoid cross-image model refitting.

Hide

```
print(sessionInfo())
```

```
## R version 4.2.1 (2022-06-23)
## Platform: x86_64-apple-darwin17.0 (64-bit)
## Running under: macOS Big Sur ... 10.16
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRblas.0.d
ylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.2/Resources/lib/libRlapack.d
ylib
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.33     R6_2.5.1          lifecycle_1.0.4   jsonlite_1.8.8
##  [5] evaluate_0.23     highr_0.10        cachem_1.0.8      rlang_1.1.2
##  [9] cli_3.6.2         rstudioapi_0.15.0 jquerylib_0.1.4   bslib_0.6.1
## [13] rmarkdown_2.25    tools_4.2.1       xfun_0.41         yaml_2.3.8
## [17] fastmap_1.1.1     compiler_4.2.1    htmltools_0.5.7   knitr_1.45
## [21] sass_0.4.8
```

References and source materials

1. Massachusetts Institute of Technology. *15.071x The Analytics Edge — Unit 6 Recitation: Seeing the Big Picture: Segmenting Images to Create Data*. Course slides (`Unit6_Recitation.pdf`).
2. Massachusetts Institute of Technology. *Unit 6 Recitation R Script* (`Unit6_Recitation.R`).
3. *Unit 6*, *Week 6.3.2 R Analysis* (`week6_3.2.R`).
4. `healthy.csv` and `tumor.csv` , accompanying grayscale MRI pixel-intensity datasets.
5. R Core Team. *R: A Language and Environment for Statistical Computing*. Functions `kmeans()` , `findInterval()` , `image()` , and `hist()` .